

Firma digital con RSA

Tabla de contenidos

¿Qué problema resuelve una firma digital?	1
Hash (huella digital) del mensaje	3
Definición matemática de la firma RSA	3
Ejemplo	4
Definimos un hash didáctico	4
Firmar	4
Verificar	4
Código Python	5
Buenas prácticas (idea general)	6
Resumen	6

Objetivos

- Entender qué garantiza una **firma digital**: autoría, integridad y no repudio.
- Conocer la **idea de RSA** y cómo se usa para firmar.
- Repasar conceptos de **aritmética modular** (potencias, inverso).
- Ver la **definición matemática** de firmar y verificar con RSA.
- Hacer un pequeño **ejemplo numérico**.

¿Qué problema resuelve una firma digital?

Una firma digital permite comprobar que:

1. El autor es quien dice ser: **autenticidad**.
2. El mensaje no ha cambiado: **integridad**.
3. El autor no puede negar haber firmado: **no repudio**.

El autor **aplica su clave privada** para firmar y cualquiera puede verificar con su **clave pública** que el mensaje ha sido firmado por su autor.

Recordatorio: aritmética modular

Trabajaremos “módulo n ”. Diremos que dos números son congruentes si dejan el **mismo resto** al dividir entre n :

$$a \equiv b \pmod{n} \iff n \mid (a - b).$$

Potencias módulo n

Para $a \in \mathbb{Z}$ y exponente $k \geq 0$, calculamos $a^k \pmod{n}$, normalmente usando **exponenciación rápida**.

Inverso modular

Si $\text{mcd}(a, n) = 1$, existe $a^{-1} \pmod{n}$ tal que:

$$a \cdot a^{-1} \equiv 1 \pmod{n}.$$

Se obtiene con el **algoritmo extendido de Euclides**.

Repaso rápido de RSA

Generamos las claves:

1. Elegir dos primos grandes p, q y $n = pq$.
2. $\varphi(n) = (p - 1)(q - 1)$.
3. Elegir e con $1 < e < \varphi(n)$ y $\text{mcd}(e, \varphi(n)) = 1$.
4. Calcular d como **inverso** de $e \pmod{\varphi(n)}$:

$$e \cdot d \equiv 1 \pmod{\varphi(n)}.$$

-
- **Clave pública:** (n, e)
 - **Clave privada:** (n, d)
-

- Cifrar un mensaje RSA: $M \equiv m^e \pmod{n}$.
- Descifrar un mensaje: $m \equiv M^d \pmod{n}$.

Hash (huella digital) del mensaje

No firmamos el texto entero directamente, sino su **huella** (hash) $H(m)$. Un buen hash es **determinista, rápido y sensible a cambios**: con un cambio mínimo en el mensaje, la huella cambia por completo.

En la práctica se usan funciones como **SHA-256**. Aquí usaremos una **huella didáctica** para el ejemplo.

Definición matemática de la firma RSA

Sea m el mensaje y $H(m)$ su huella (un entero $0 \leq H(m) < n$).

- **Firma** con la clave privada (n, d) :

$$s \equiv H(m)^d \pmod{n}.$$

- **Verificación** con la clave pública (n, e) :

$$v \equiv s^e \pmod{n}.$$

Aceptamos la firma si

$$v \equiv H(m) \pmod{n}.$$

Idea clave: elevar con d y luego con e “revierte” gracias a

$$e \cdot d \equiv 1 \pmod{\varphi(n)}$$

y al teorema de Euler.

En la práctica se añade un **relleno (padding) seguro** (por ejemplo, **RSA-PSS**) antes de elevar, para evitar ataques. Aquí lo omitimos por ser introducción.

Ejemplo

! Importante

En clase usaremos números **pequeños** para entender el proceso. En la realidad p y q deben ser **muy grandes** (la longitud de n debe ser al menos de 2048 bits).

- Elegimos $p = 61$, $q = 53 \Rightarrow n = pq = 3233$.
- $\varphi(n) = (61 - 1)(53 - 1) = 3120$.
- Elegimos $e = 17$ con $\text{mcd}(17, 3120) = 1$.
- Hallamos d tal que $17 \cdot d \equiv 1 \pmod{3120}$. Se obtiene $d = 2753$.

Claves:

- Pública $(n, e) = (3233, 17)$
- Privada $(n, d) = (3233, 2753)$

Definimos un hash didáctico

Sea

$$H(m) = \left(\sum \text{códigos ASCII de los caracteres de } m \right) \bmod n.$$

Para el mensaje $m = \text{"OK"}$: ASCII 'O' = 79, 'K' = 75. Luego $H(m) = (79 + 75) \bmod 3233 = 154$.

Firmar

$$s \equiv H(m)^d \equiv 154^{2753} \pmod{3233}.$$

(Se calcula por exponenciación modular rápida.)

Verificar

$$v \equiv s^e \equiv (154^{2753})^{17} \equiv 154 \pmod{3233},$$

que coincide con $H(m)$.

Código Python

```
# Ejemplo didáctico de firma RSA con números pequeños.

def egcd(a, b):
    """Algoritmo extendido de Euclides: devuelve (g, x, y) tal que ax + by = g = gcd(a,b)."""
    if b == 0:
        return (a, 1, 0)
    g, x1, y1 = egcd(b, a % b)
    return (g, y1, x1 - (a // b) * y1)

def modinv(a, m):
    """Inverso modular de a mod m (si gcd(a,m)=1)."""
    g, x, _ = egcd(a, m)
    if g != 1:
        raise ValueError("No existe inverso")
    return x % m

def hash(msg, n):
    """Hash didáctico: suma de códigos ASCII mod n (solo para clase)."""
    return sum(ord(ch) for ch in msg) % n

# Parámetros didácticos
p, q = 61, 53
n = p * q          # 3233
phi = (p-1)*(q-1)  # 3120
e = 17
d = modinv(e, phi) # 2753

m = "OK"
h = hash(m, n)

# Firma
s = pow(h, d, n)

# Verificación
v = pow(s, e, n)

print("n =", n, "phi =", phi, "e =", e, "d =", d)
print("Mensaje:", m, "Hash:", h)
print("Firma s:", s)
print("Verificación v:", v, "=>", "válida" if v == h else "inválida")
```

Buenas prácticas (idea general)

- **Padding seguro:** en la práctica se usa **RSA-PSS** (Probabilistic Signature Scheme) para firmar.
- **Hash criptográfico:** SHA-256 o superior.
- **Tamaño de clave:** n de **2048 bits** o más.
- **Gestión de claves:** proteger la clave **privada**; distribuir bien la clave **pública**.

Resumen

- Una firma RSA se define como:

$$s \equiv H(m)^d \pmod{n}.$$

- Se verifica con:

$$v \equiv s^e \pmod{n}.$$

- Si $v \equiv H(m) \pmod{n}$, la firma es válida (mensaje íntegro y autor auténtico).
- En la vida real, se añaden **hash seguro** y **padding (RSA-PSS)** para evitar ataques.